

บทที่ 10

เอนจินของกล้องระดับที่ 2

ในคลาส cCamera ที่ได้จัดทำขึ้นในเอนจิน ถือเป็นคลาสที่มีความสามารถในการจัดการเรื่องกล้องที่สืบทอดมาจากไคลเร็กเอนจินได้ระดับหนึ่ง ซึ่งมีความสามารถในการย้าย (Move) , การหมุน (Rotation) แต่ในการเคลื่อนไหวของกล้องที่จะเอามาใช้ในเกมจริงๆ นั้น สามารถพัฒนาให้ใช้งานง่ายขึ้นไปอีก ตามแบบอย่างของเกมที่จะสร้างขึ้นมา ดังนั้น จึงมีการนำเอา คลาส cCamera ซึ่งใช้พื้นฐานของไคลเร็กเอนจินมาพัฒนาต่อออกเป็น คลาส cCameraLv2 เพื่อที่จะได้อาเอนจินที่กล่าวมานี้ไปใช้ใน เกมที่จะสร้างขึ้นมาได้

เนื่องจากการเคลื่อนไหวของกล้องนั้น แบ่งได้เป็น 3 แบบ หลักๆ ดังนั้น การที่จะทำให้กล้องเคลื่อนไหวได้อย่างมีประสิทธิภาพนั้น จึงจำเป็นต้องรู้ชนิดของการเคลื่อนไหวก่อน ซึ่งการเคลื่อนไหวนั้น แบ่งออกเป็น 3 ชนิดดังนี้

1. First – Person View
2. Free – Look Camera
3. Camera around Object

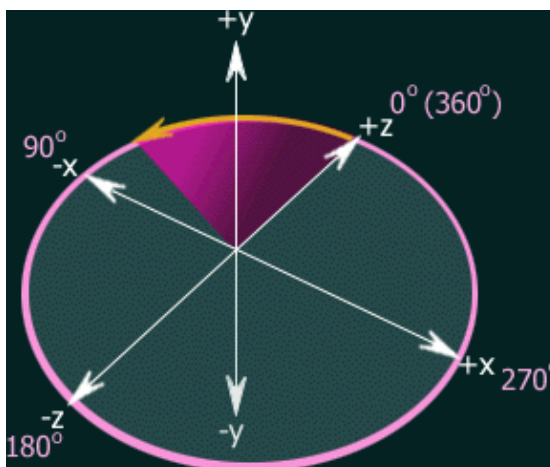
10.1 First – Person View

First – Person View หรือ การเคลื่อนไหวกล้องแบบ “มุมมองบุคคลที่ 1” นั้น จะเป็นการเคลื่อนที่ของกล้อง โคนวางตำแหน่งของกล้องไว้ ณ.ที่จุดดวงตาของตัวละคร ดังนั้น เราจะเห็นในสิ่งที่ตัวละครเห็น เมื่อตัวละครเคลื่อนที่ไปข้างหน้า กล้องก็จะเคลื่อนที่ตามไปด้วย ซึ่งโดยส่วนมาก มุมมองบุคคลที่ 1 นั้น จะไม่สามารถมองเห็นตัวละครที่เรา กำลังเล่นได้ ดังนั้น จึงเขียนโค้ดเพื่อรับค่าจากเมาส์และคีย์บอร์ด แล้วสั่งให้กล้องเคลื่อนไหวโดยตรงตามที่เรต้องการเลย โดยการสั่งให้กล้องเคลื่อนไหวนั้นจะมีความสามารถดังนี้

- หมุนกล้องไปทางซ้ายและขวา (อาจใช้โดยการเลื่อนเมาส์ไปทางซ้ายและขวาตามลำดับ)
- แขนงมองขึ้นข้างบนและมองลงข้างล่างได้ (อาจใช้โดยการเลื่อนเมาส์ไปข้างบนและข้างล่างตามลำดับ) นอกจากนี้การมองไปด้านบนและด้านล่างนั้นต้องกำหนดให้มุมของการมองไม่สามารถเกิน 90 องศาได้ คือสามารถมองขึ้นได้สูงสุดคือที่เหนือหัวของตัวเองเท่านั้น ไม่สามารถที่จะมองเอนไปข้างหลังได้ และจุดที่มองต่ำสุดคือเท้าของตัวเองหรือตำแหน่งที่กำลังยืนอยู่
- เคลื่อนที่ไปข้างหน้า , เคลื่อนที่ไปข้างหลัง , ขยับไปทางซ้าย , ขยับไปทางขวา (อาจใช้โดยการกดปุ่มบน คีย์บอร์ด ตามที่กำหนด)

10.1.1 การหมุนกลิ้งไปทางซ้ายและขวา

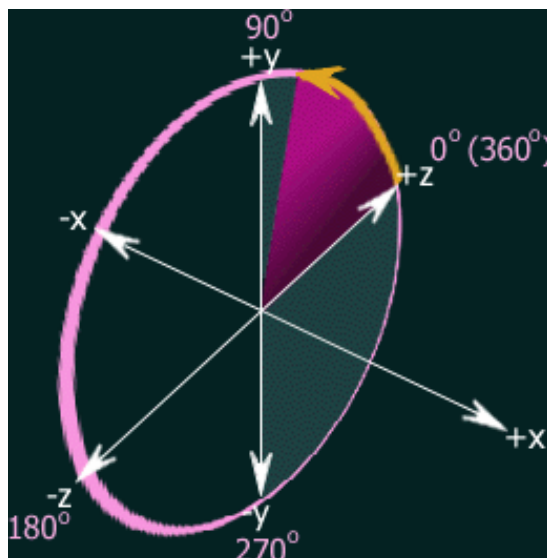
การหมุนกลิ้งไปทางซ้ายและขวานั้น (หรือเรียกอีกอย่างว่าการ Yaw) ในโลกสามมิติ คือการหมุนรอบแกน y นั่นเอง ดังนั้น เราต้องมีการสั่งไว้ว่า ให้หมุนไปกี่องศาโดยเก็บค่าของการหมุนไว้ (ใน คลาส ใช้ตัวแปรเป็นชนิด float ชื่อว่า $fAngleY$) ประกติแล้ว การหมุนรอบแกน Y จะเป็นการหมุนแบบทวนเข็มนาฬิกา ดังนั้น จึงจะรับค่าองศาของการหมุนแต่ละทีตามค่าแรงเคลื่อนที่ของเมาส์กล่าวคือ ถ้าเมาส์เคลื่อนที่เร็ว แรง องศาที่จะเพิ่มเข้าไปก็จะมีมาก และจะหมุนได้เร็ว เมื่อได้ค่าแรงการเคลื่อนที่ของเมาส์ แล้วก็นำมาบวกให้แก่ตัวแปร $fAngleY$ ไปเรื่อยๆ แต่เนื่องจากมุมของการหมุนนั้น อยู่ในช่วง 0.0 ถึง 360.0 องศา ดังนั้น จึงต้องกำหนดให้ตัวโปรแกรมเช็คตรงส่วนนี้ไม่ให้ค่า $fAngleY$ มีน้อยไปกว่า 0.0 และไม่มากไปกว่า 360.0 องศา สุดท้ายคือการสั่งให้กล้องหมุนตามค่า $fAngleY$ นั่นเอง (การหมุนกลิ้งจะใช้เมธอดของคลาส $cCamera$ ในการหมุน)



รูปที่ 10-1 การหมุนทวนเข็มนาฬิกาของมุมรอบแกน y

10.1.2 การแหงนมองขึ้นข้างบนและมองลงข้างล่าง

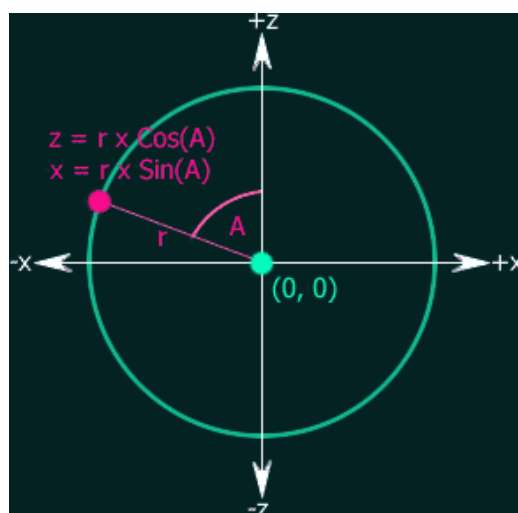
ส่วนการแหงนมองขึ้นข้างบนและมองลงข้างล่าง (หรือเรียกอีกอย่างว่า Pitch) นั้นจะเป็นการหมุนรอบแกน x เราจึงทำเหมือนการหมุนรอบแกน y คือเก็บค่าองศาของการหมุนไว้ (ในคลาสใช้ตัวแปรที่ชื่อ $fAngleX$) โดยการหมุนรอบแกน x มันจะหมุนแบบทวนเข็มนาฬิกา เราจึงใช้ความแรงในการเลื่อนเมาส์มาเพิ่มหรือลบให้กับค่าของตัวแปร $fAngleX$ เหมือนกับการหมุนรอบแกน y แต่การหมุนนั้น ต้องกำหนดมุมเอนขึ้นและเอนลงด้วย ดังนั้นค่าขององศาที่จะรับได้ จะอยู่ที่ 0.0 ถึง 90.0 องศา (มองขึ้น) และ 270.0 ถึง 360.0 องศาแล้วจึงสั่งให้กล้องหมุนตามองศาของ $fAngleX$ เพียงเท่านี้ กล้องก็จะสามารถหมุนขึ้นบนลงล่างได้แล้ว



รูปที่ 10-2 การหมุนทวนเข็มนาฬิกาของมุมรอบแกน X

10.1.3 การเคลื่อนที่ในแนวระนาบ (ซ้าย , ขวา , หน้า , หลัง)

การเคลื่อนที่ในแนวระนาบของมุมมองบุคคลที่ 1 นั้น จะเป็นการเคลื่อนที่ในแกนแนว X และ แกน Z ของเวอร์สเปส (World Space) (เพราะแกน X และแกน Z นั้นตั้งฉากกันอยู่ในแนวระนาบ) โดย แกน Z จะเป็นการเคลื่อนที่เข้าหาหน้าจอ แกน -Z พุ่งไปด้านหลัง แก X อยู่ทางซ้าย และแกน -X อยู่ทาง ด้านขวา ซึ่งในการเคลื่อนที่นั้น ไม่ใช่ว่าถ้าให้เดินหน้า แล้วเราจะสามารถบอกค่าคงที่เข้าไปในแนวแกน Z ได้เลย เพราะการทำอย่างนั้น จะถูก ก็ต่อเมื่อมุมมองหันเข้าหาแกน +Z ของเวอร์สเปสตลอดเวลาเท่านั้น ถ้าเราหันไปทางอื่น เช่น หันไปทางขวา พอเวลาเดินหน้าแล้วเรา + ค่าไปแกแกน Z จะเห็นได้ว่าตัวเราจะ พุ่งไปทางซ้ายบน ไม่ใช่พุ่งเดินหน้าไปตรงๆเลย ดังนั้น จึงต้องมีการเช็คการหมุนตัว จึงต้องมีการเพิ่มและ ลดค่าของทั้งสองแกนพร้อมๆกันตามองศาของการหันหน้าดังรูปที่ 10-3



รูปที่ 10-3 การปรับเปลี่ยนค่าของแกน XZ

การเคลื่อนที่ในขณะที่หมุนนั้น จากรูปจะเห็นวงกลมที่มีจุดศูนย์กลาง (Origin) อยู่ที่ตำแหน่ง (0, 0) มีรัศมี r หน่วยวิ่งไปโดยมีมุม A ดังนั้น

- ตำแหน่งของ $z = r \times \cos(A)$
- ตำแหน่งของ $x = r \times \sin(A)$

โดยที่ตำแหน่ง (0, 0) นั้นคือตำแหน่งของกล้อง r คือระยะทางที่เราต้องการเคลื่อนที่ไปภายใน 1 ครั้ง (ความเร็วของการเคลื่อนที่กล้อง) และมุม A คือมุมของการหมุนรอบแกน Y หรือมุม $fAngleY$ นั่นเอง

ส่วนการเคลื่อนที่ไปด้านข้างทางซ้ายนั้น เพียงแค่บวกมุมตามเข็มนาฬิกาไป อีก 90 องศา (ทำให้ตั้งฉากกับการเคลื่อนที่ไปข้างหน้า) แล้วจึงคำนวณแบบเดียวกับการเคลื่อนที่ไปข้างหน้า จึงจะได้สูตรว่า

- ตำแหน่งของ $z = r \times \cos(A+90)$
- ตำแหน่งของ $x = r \times \sin(A+90)$

ในการทำงานเดียวกัน การเคลื่อนที่ไปทางด้านขวานั้น เพียงแต่ต่างกันตรงมุมที่จะบวกเข้าไป คือ ให้หันไปอีกด้านต้องลบด้วยมุม 90 องศา (หรือจะบวกเข้าไปอีก 270 องศาก็ได้) จึงได้สูตรว่า

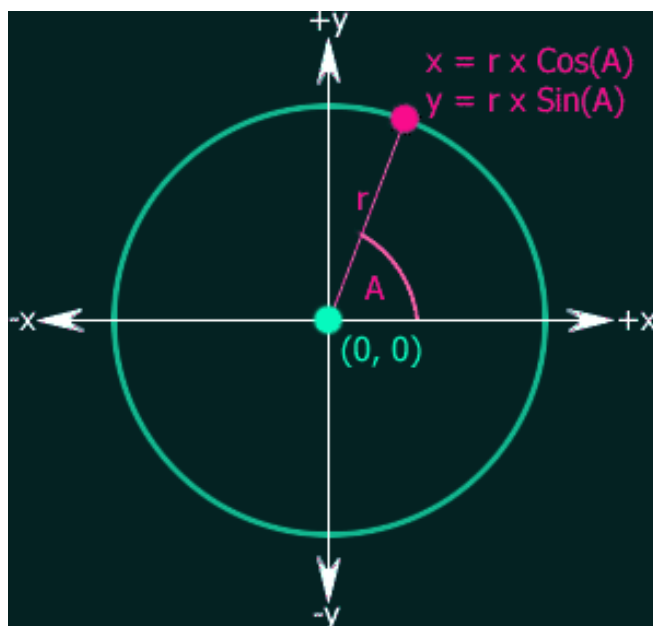
- ตำแหน่งของ $z = r \times \cos(A-90)$
- ตำแหน่งของ $x = r \times \sin(A-90)$

เมื่อเราได้จุด x และ z ที่เราจะเคลื่อนที่ไปแล้ว ต่อไปจึงเอาจุดนั้น ไปวางไว้ที่ตำแหน่ง Eye – Point เพื่อที่จะให้กล้อง เคลื่อนที่ไปยังทิศทางการควบคุมดังที่เราได้กำหนดไว้ ส่วน Look – At Point ก็เพียงแค่บวกตำแหน่งของกล้องในแนว Z เข้าไปเท่านั้น เพราะกล้องต้องเล็งไปข้างหน้าหรือเข้าหาจอภาพตลอดเวลา สำหรับทิศทางการตั้งของกล้องจะตั้งขึ้นในแนวแกน $+y$ ตลอด และเนื่องจากการเดินในแนวระนาบ จึงไม่มีค่าความสูง (Y) เข้ามาเกี่ยวข้องใน Eye – Point และ Look – At Point เลย

10.2 Free – Look Camera

Free – Look Camera หรือมุมกล้องแบบอิสระคือการเคลื่อนที่แบบอิสระที่ทำให้เราสามารถบินไปได้ทั่วฉาก สามารถเคลื่อนที่ไปยังจุดไหนก็ได้ภายในเวอร์สเปสโดยใช้เมาส์และคีย์บอร์ดในการเคลื่อนที่ของกล้อง

จะเห็นได้ว่า Free – Look Camera ก็จะเหมือนการเอามุมกล้องบุคคลที่ 1 มาคำนวณเพิ่มเติมในด้านของความสูงกล่าวคือ มุมกล้องบุคคลที่ 1 สามารถเคลื่อนที่ได้ในระนาบ XZ โดยมีมุมหมุนรอบแกน Y มากำหนดทิศทาง แต่ Free – Look Camera จะสามารถเคลื่อนที่ได้ในระนาบ XYZ โดยมีมุมหมุนรอบแกน X และ Y มาช่วยคอยกำหนดทิศทาง



รูปที่ 10-4 การปรับแก้ค่าของแกน XY

จากรูปที่ 10-4 จะเห็นได้ว่า ถ้ามีการมองเงาขึ้นมา ซึ่งมุม B (ในรูปคือ A) เป็นมุมที่หมุนรอบแกน Z โดยมีรัศมี r เป็นความยาว จากสูตรของมุมมองบุคคลที่ 1 จึงสามารถเปลี่ยนเป็นสูตรของมุมกล้องแบบอิสระได้ว่า

- ตำแหน่งของ $z = r \times \cos(A) \times \cos(B)$
- ตำแหน่งของ $x = r \times \sin(A) \times \cos(B)$
- ตำแหน่งของ $y = r \times \sin(A) \times \sin(B)$

เมื่อเราได้จุด x และ z ที่เราจะเคลื่อนที่ไปแล้ว ต่อไปจึงเอาจุดนั้น ไปวางไว้ที่ตำแหน่ง Eye - Point เหมือนการเคลื่อนที่แบบบุคคลที่ 1 และ Look - At Point ก็ทำเหมือนกันคือบวกตำแหน่งของกล้องในแนว Z เข้าไปเท่านั้น เพราะกล้องต้องเล็งไปข้างหน้าหรือเข้าหาจอภาพตลอดเวลา และเนื่องจากเป็นการเดินในแนวอิสระ จึงไม่มีค่าความสูง (Y) เข้ามาเกี่ยวข้องใน Eye - Point และ Look - At Point ต่างกับการเคลื่อนที่แบบบุคคลที่ 1 ซึ่งจะ Fix ค่า Y ไว้เลย

10.3 Camera around Object

Camera around Object หรือการเคลื่อนกล้องรอบวัตถุ การเคลื่อนที่แบบนี้ จะเหมือนการที่เราเอากล้อง ถ่ายไปยังวัตถุที่เราสนใจ แล้วหมุนกล้องเพื่อให้ได้มุมต่างๆตามที่เราต้องการโดยที่จุดที่กล้องส่องไปจะเป็นจุดกึ่งกลางของวัตถุนั้นเสมอ ไม่ว่าวัตถุนั้นจะเคลื่อนที่ไปทางไหนก็ตาม กล้องจะเคลื่อนที่ตามไปด้วย ดังนั้น เราจะเห็นวัตถุนั้นอยู่นิ่ง แต่ฉากรอบข้างจะเปลี่ยนไป นั่นเป็นเพราะ กล้องของเราเคลื่อนที่ไปพร้อมกับวัตถุนั่นเอง

ในส่วนของการควบคุมนั้น เนื่องจากการเคลื่อนที่ของกล้องจะเคลื่อนที่ไปตามตำแหน่งของวัตถุ อยู่แล้วเราจึงไม่จำเป็นต้องมีการควบคุมการเคลื่อนที่ของกล้อง แต่จะเป็นการควบคุมการหมุนรอบวัตถุซะมากกว่า ซึ่งการหมุนรอบวัตถุนั้น สามารถควบคุมได้โดยการรับมุมเข้ามาเหมือนองศาของมุมกล้องแบบอิสระ แล้วนำมาคำนวณเพื่อหาค่า ตำแหน่งที่กล้องจะเคลื่อนที่ไปโดยใช้สูตรเหมือนกับมุมกล้องแบบอิสระคือ

- ตำแหน่งของ $z = r \times \cos(A) \times \cos(B)$
- ตำแหน่งของ $x = r \times \sin(A) \times \cos(B)$
- ตำแหน่งของ $y = r \times \sin(B)$

แต่เมื่อเราจะเอาตำแหน่งนั้นเข้าสู่ กล้อง จะต่างกันตรงที่ว่าตอนนี้จุดที่มอง (Look – At Point) จะไม่ใช่มองไปข้างหน้า หรือบอกตำแหน่งของกล้องในแนวแกน Z แต่จะเป็นจุดที่วัตถุอยู่ เพื่อที่จะทำให้กล้องของเรา ถ่ายไปยังวัตถุนั้นๆ ส่วนตำแหน่งที่วางกล้อง ก็ได้จากค่าตำแหน่งของ XYZ ที่เราหาจากมุม A และ B แต่เนื่องจากกล้องต้องเคลื่อนที่ตามวัตถุ เราจึงจำเป็นต้องบอกตำแหน่งของวัตถุที่เรามองอยู่เข้าไปด้วย กล้อง จึงจะสามารถเคลื่อนที่ตามวัตถุไปได้

10.4 คลาส cCameraV2

คลาส cCameraV2 นั้น เป็นคลาสที่ดึงเอาความสามารถของคลาส cCamera มาเพื่อให้มีการคำนวณการเคลื่อนไหวของกล้องทั้ง 3 แบบให้ใช้งานได้ง่ายขึ้นโดยผู้ใช้ไม่ต้องมาคำนวณว่าตำแหน่ง x, y, z และมุมที่ทำกับแกน x, y, z จะมีค่าเป็นเท่าไร เพียงแค่เช็คว่าเมื่อไรรับค่าจากเมาส์หรือคีย์บอร์ด แล้วให้เรียกว่าจะให้กล้องหมุนซ้าย, หมุนขวา, เดินหน้า, ถอยหลัง เพียงแค่นี้ในคลาส cCameraV2 ก็จะคำนวณตำแหน่ง x, y, z และมุมรอบแกน x, y, z ให้และส่งค่านั้นไปยังคลาส cCamera เพื่อนำกล้องไปวางตามตำแหน่งที่เราต้องการต่อไป

ในส่วนของค่าต่างๆในคลาส จะมีการเก็บค่ามุมที่ทำกับแนวแกน XYZ ตำแหน่งกล้องในแนวแกน XYZ เก็บค่าตำแหน่งของวัตถุที่กล้องจะจับไว้ในแนวแกน เก็บค่าความเร็วของการเคลื่อนที่และการหมุนของคีย์บอร์ดและเมาส์ และเก็บการคำนวณของการเคลื่อนไหวที่เราต้องการจะให้คำนวณ

cCameraLv2	
 m_Camera : cCamera	
 fAngleX : Float	
 fAngleY : Float	
 fPosX : Float	
 fPosY : Float	
 fPosZ : Float	
 flookX : Float	
 flookY : Float	
 flookZ : Float	
 KEYBOARD_ROTATION_SPEED : Float	
 KEYBOARD_MOVING_SPEED : Float	
 MOUSE_ROTATION_SPEED : Float	
 MOUSE_MOVING_SPEED : Float	
 distance : Float	
 movetype : Integer	
 setspeed(krs : Float, kms : Float, mrs : Float, mms : Float) : Void	
 setfirstxyz(x : Float, y : Float, z : Float) : Void	
 setlookatpoint(x : Float, y : Float, z : Float) : Void	
 setdistance(dist : Float) : Void	
 setmovetype(mtype : Integer) : Void	
 moveforward() : Void	
 moveback() : Void	
 moveleft() : Void	
 moveright() : Void	
 lookupdown(lup : Long) : Void	
 lookleftright(lir : Long) : Void	
 calcamera() : Void	
 resetpos() : Void	
 getposX() : Float	
 getposY() : Float	
 getposZ() : Float	
 getlookX() : Float	
 getlookY() : Float	
 getlookZ() : Float	

รูปที่ 10-5 คลาสไดอะแกรมของ cCameraLv2

แต่ละเมธอดมีรายละเอียดดังนี้

- setspeed() ใช้ในการกำหนดความเร็วของการหมุน และการเคลื่อนที่ให้กับคีย์บอร์ดและเมาส์
- setfirstxyz() ใช้ในการกำหนดค่าเริ่มต้น ให้กับตำแหน่งของกล้องในเวอร์สเปส
- setlookatpoint() ใช้ในการกำหนดค่าที่กล้องจะมองหรือตำแหน่งในเวอร์สเปสของวัตถุที่เราต้องการจะติดตามในมุมมองแบบ Camera around Object
- setdistance() ใช้ในการกำหนดค่าความห่างระหว่างกล้องกับวัตถุที่เราจะหมุนรอบ
- setmovetype() กำหนดชนิดของการเคลื่อนที่ว่าจะให้เคลื่อนที่แบบไหน โดยที่
 - 0 = First – Person View
 - 1 = Free – Look Camera
 - 2 = Camera around Object

- `moveforward()`, `moveback()`, `moveleft()`, `moveright()` ให้กล้องเคลื่อนที่ไปตามความเร็วที่กำหนดไว้ในเมธอด `setspeed`
- `lookupdown()` ตั้งให้กล้องหมุนซ้ายหรือขวาตามความเร็วที่กำหนดไว้ในเมธอด `setspeed` และมีการรับค่าจากเมาส์มาเพื่อบอกแรงของการหมุนด้วย
- `lookleftright()` ให้คำนวณการเคลื่อนไหวของจุด `X`, `Y`, `Z` ตามชนิดของการเคลื่อนที่ของกล้อง (`Movetype`) และจึงนำค่าเหล่านั้น เรียกใช้ในเมทอดที่สืบทอดมาจาก คลาส `cCamera` ของตัวแปร `m_Camera` นั้นเอง
- `calcamera()` ให้คำนวณการเคลื่อนไหวของจุด `X`, `Y`, `Z` ตามชนิดของการเคลื่อนที่ของกล้อง (`Movetype`) และจึงนำค่าเหล่านั้น เรียกใช้ในเมทอดที่สืบทอดมาจาก คลาส `cCamera` ของตัวแปร `m_Camera` นั้นเอง
- `resetpos()` ตั้งให้ตำแหน่งต่างๆ ของกล้อง กลับมาอยู่ที่จุดเริ่มต้น
- `getposX`, `getposY`, `getposZ` ในการรีเทริน (return) ค่า `fPosX`, `fPosY`, `fPosZ` ตามลำดับ
- `getlookX`, `getlookY`, `getlookZ` ใช้ในการรีเทรินค่า `flookX`, `flookY`, `flookZ` ตามลำดับ